Q1. CORS (Cross-Origin Resource Sharing)

Meaning:

CORS is a security mechanism implemented by browsers that restricts web pages from making requests to a different domain than the one that served the web page. It's a way to relax the **Same-Origin Policy**.

Why Required:

- **Security**: Prevents malicious websites from accessing sensitive data from other sites
- **Cross-domain APIs**: Allows legitimate frontend apps (different origins) to consume backend APIs
- **Control**: Gives servers control over which origins can access their resources

How to Configure:

Backend (Node.js/Express):

javascript

```
const express = require('express');
const cors = require('cors');
const app = express();

// Enable CORS for specific origin
app.use(cors({
  origin: 'http://localhost:3000', // React app URL
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization']
}));

// Or allow all origins (development only)
app.use(cors());
```

Backend (Spring Boot):

java

```
@CrossOrigin(origins = "http://localhost:3000")
@RestController
@RequestMapping("/api/products")
public class ProductController {
```

```
    // Controller methods
}
```

Q2. React CRUD Operations for Product

jsx

```
// App.js - Main Component
import React, { useState, useEffect } from 'react';
import axios from 'axios';

const API_URL = 'http://localhost:8080/api/products';

function App() {
  const [products, setProducts] = useState([]);
  const [formData, setFormData] = useState({ name: '', price: '',
description: '' });
  const [editingId, setEditingId] = useState(null);

  // Fetch all products on component mount
  useEffect(() => {
    fetchProducts();
  }, []);

  // GET - Fetch all products
  const fetchProducts = async () => {
    try {
      const response = await axios.get(API_URL);
      setProducts(response.data);
    } catch (error) {
      console.error('Error fetching products:', error);
    }
  };

  // POST - Add new product
  const addProduct = async (e) => {
    e.preventDefault();
    try {
      const response = await axios.post(API_URL, formData);
      setProducts([...products, response.data]);
      resetForm();
    } catch (error) {
```

```
      console.error('Error adding product:', error);
    }
  };

  // PUT - Update existing product
  const updateProduct = async (e) => {
    e.preventDefault();
    try {
      const response = await axios.put(`${API_URL}/${editingId}`,
formData);
      setProducts(products.map(p => p.id === editingId ? response.data :
p));
      resetForm();
    } catch (error) {
      console.error('Error updating product:', error);
    }
  };

  // DELETE - Remove product
  const deleteProduct = async (id) => {
    if (window.confirm('Are you sure?')) {
      try {
        await axios.delete(`${API_URL}/${id}`);
        setProducts(products.filter(p => p.id !== id));
      } catch (error) {
        console.error('Error deleting product:', error);
      }
    }
  };

  // Fill form for editing
  const editProduct = (product) => {
    setEditingId(product.id);
    setFormData({ name: product.name, price: product.price, description:
product.description });
  };

  // Reset form
  const resetForm = () => {
    setFormData({ name: '', price: '', description: '' });
    setEditingId(null);
```

```
  };

  // Handle form input changes
  const handleChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };

  return (
    <div style={{ padding: '20px' }}>
      <h1>Product Management</h1>

      {/* Add/Edit Form */}
      <form onSubmit={editingId ? updateProduct : addProduct}>
        <input
          name="name"
          placeholder="Product Name"
          value={formData.name}
          onChange={handleChange}
          required
        />
        <input
          name="price"
          placeholder="Price"
          type="number"
          value={formData.price}
          onChange={handleChange}
          required
        />
        <input
          name="description"
          placeholder="Description"
          value={formData.description}
          onChange={handleChange}
          required
        />
        <button type="submit">
          {editingId ? 'Update' : 'Add'} Product
        </button>
        {editingId && <button onClick={resetForm}>Cancel</button>}
      </form>
```

```
      {/* Products List */}
      <table border="1" cellPadding="10">
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Price</th>
            <th>Description</th>
            <th>Actions</th>
          </tr>
        </thead>
        <tbody>
          {products.map(product => (
            <tr key={product.id}>
              <td>{product.id}</td>
              <td>{product.name}</td>
              <td>${product.price}</td>
              <td>{product.description}</td>
              <td>
                <button onClick={() =>
editProduct(product)}>Edit</button>
                <button onClick={() =>
deleteProduct(product.id)}>Delete</button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

export default App;
```

Package.json dependencies:

json

```
{
  "dependencies": {
    "react": "^18.2.0",
    "axios": "^1.6.0"
```

```
        }
      }
```

Key Points:

- Uses `axios` for HTTP requests
- Supports all CRUD operations (GET, POST, PUT, DELETE)
- Form handles both add and edit modes
- State management with React hooks